

Lab 08 — Cost Optimisation — Storage & Compute

Measure what is actually consumed, spot idle capacity, move data to an archive tier, and right-size a service.

Lab environment: Docker Desktop (free local simulation) — runs locally, no AWS account needed

Estimated time: 35–40 minutes

SAA-C03 domain: Design Cost-Optimized Architectures

Overview

Local component	AWS equivalent
<code>docker stats</code>	Amazon CloudWatch metrics / AWS Compute Optimizer
<code>lab8-app</code> memory limit (256 MB → 128 MB)	Right-sizing an instance or task
<code>lab8-store</code> hot → archive folders (MinIO)	S3 lifecycle transition to an archive storage class

No ports are published for this lab — every exercise uses `docker exec` or `docker stats`.

Step 1 — Get the lab source and start the lab

The repo lives in `~/Downloads/aws-saa`. The paths are absolute, so these commands work from any folder. `git clone` is needed only once: if the repo is already there it prints `already exists`, and the next line still takes you into this lab.

```
cd ~/Downloads
git clone https://github.com/mohanpothula/aws-saa.git
cd ~/Downloads/aws-saa/labs/lab-08-cost-optimisation
docker compose up -d
docker compose ps
```

Step 2 — Find resource usage

Task: Inspect running container consumption.

```
docker stats --no-stream
```

Expected result: CPU and memory usage are shown for each container.

Step 3 — Identify waste

Task: Find containers with low utilisation.

```
docker stats --no-stream
```

Expected result: You identify the low-use service and its unused headroom.

Step 4 — Storage lifecycle

Task: Create hot and archive folders and move an old object.

```
docker exec lab8-store sh -c 'mkdir -p /data/hot /data/archive && echo old >/data/hot/old.txt
&& mv /data/hot/old.txt /data/archive/'
```

Expected result: The object has moved to `/data/archive/`.

Step 5 — Right-size the service

Task: Edit the memory limit in `docker-compose.yml` from `256M` to `128M`, then recreate.

```
docker compose up -d --force-recreate
docker inspect lab8-app --format '{{.HostConfig.Memory}}'
```

Expected result: 134217728 — the new 128 MB limit.

Step 6 — Cost decision

Task: Choose always-on or on-demand for an infrequent job.

```
echo 'Write choice and reason'
```

Expected result: An on-demand / event-driven model for the infrequent workload, with the reason.

Verification

Check	Expected
Step 2 — Find resource usage	CPU and memory usage are shown for each container
Step 3 — Identify waste	You identify the low-use service and its unused headroom
Step 4 — Storage lifecycle	The object has moved to <code>/data/archive/</code>
Step 5 — Right-size the service	134217728 — the new 128 MB limit
Step 6 — Cost decision	An on-demand / event-driven model for the infrequent workload, with the reason

Completion check

- [] Every validation result demonstrated to the trainer
- [] You can say which AWS service each local component represents
- [] You can justify the right-sizing change with the numbers from Step 2

Cleanup

```
docker compose down -v
```

-v also removes the lab's volumes, so the next run starts from a clean state.

Reference

- Lab source code: <https://github.com/mohanpothula/aws-saa/tree/main/labs/lab-08-cost-optimisation>
- Original student guide: [courseware/AWS_SAA_Lab8_Student_Guide.docx](#)
- [Well-Architected Cost Optimization Pillar](#)
- [AWS Compute Optimizer](#)
- [S3 lifecycle management](#)
- [Amazon S3 storage classes](#)
- [Compose file reference — deploy / resource limits](#)
- [MinIO container documentation](#)